

Spring Boot Hour 2 🧐🧐🧐 Request Journey & REST

📖 Spring Boot — 8-Hour Plan · HOUR 2

The Request Journey: Architecture + REST APIs

Goal: when the interviewer says "walk me through what happens when a request hits your API," you narrate it like a story you've personally watched happen. This is one of the top-3 most asked Spring Boot questions.

2.1 A request is born — follow it like a parcel

You hit Enter in Postman:

```
GET http://localhost:8080/api/users/1
```

What happens inside your Spring Boot app, step by step:

Step 1 — The embedded Tomcat receives it. Remember Hour 1: Tomcat lives *inside* your JAR. It's the front gate of the building — it accepts the raw HTTP request from the network.

Step 2 — Tomcat hands it to the DispatcherServlet. This is THE most important component of this hour. The **DispatcherServlet** is a single servlet that receives **every single request** to your application, and its only job is to figure out: "*who should handle this?*"

This design is called the **Front Controller pattern** — ONE central entry point in front of everything.

Analogy: a hospital reception desk. You don't wander the corridors looking for the right doctor. EVERYONE first goes to reception; reception looks at your problem ("tooth pain") and routes you to the right room ("Dentist, Room 12"). DispatcherServlet is that reception desk. Every patient (request), no exceptions, passes through it.

Step 3 — It finds the right controller method. The DispatcherServlet consults its **HandlerMapping** — a registry built at startup that says "URL pattern `/api/users/{id}` with method GET → `UserController.getUser()` ." (This registry got built when Spring scanned your `@GetMapping` annotations — Hour 1's component scan at work.)

Step 4 — Controller → Service → Repository → Database (the layers — next section).

Step 5 — The return trip. Your method returns a `User` object. Spring converts it to **JSON** automatically (a library called Jackson does this — object → JSON is called *serialization*), the DispatcherServlet wraps it in an HTTP response, Tomcat ships it back to Postman.

The full picture (page 2 of your handbook — now you know what each arrow *means*):

```
Postman → Tomcat → DispatcherServlet → Controller → Service → Repository → DB
    ↳ JSON    ↳ HTTP    ↳ (Jackson: object→JSON) ↳ User object ↳ ↳ row from table
```

🗣️ INTERVIEW CHECKPOINT — asked in almost every Spring interview

Q: "What is the DispatcherServlet?" 🧐: "*It's the front controller of Spring MVC — a single servlet that receives every request, uses handler mappings to route it to the right controller method, and handles the response rendering on the way back. It's auto-configured by Spring Boot.*"

Q: "Explain the flow when a request hits a Spring Boot application." ← the big one 🧐: "*Embedded Tomcat accepts the HTTP request and passes it to the DispatcherServlet. It looks up the handler mapping to find the matching controller method, the controller delegates to the service layer for business logic, the service calls the repository for data access, the result flows back, Jackson serializes the returned object to JSON, and the DispatcherServlet sends the HTTP response.*" Practice saying this ONE paragraph out loud 3 times. It's a guaranteed question.

2.2 Why three layers? (Controller / Service / Repository)

You *could* write everything — request handling, business rules, database queries — inside one giant controller. It would run. So why split?

Each layer has ONE job:

Layer	Its ONLY job	Must NOT do
Controller	Receive request, validate shape, return response	Business logic, DB queries
Service	Business logic and rules ("age must be ≥18 to register", "deduct balance then notify")	Know anything about HTTP or SQL
Repository	Talk to the database — CRUD	Business decisions

Analogy: a restaurant. The **waiter** (controller) takes your order and brings your food — he never cooks. The **chef** (service) cooks and decides "this dish needs more salt" — he never meets customers. The **storekeeper** (repository) fetches ingredients from the storage room — he neither cooks nor serves. Imagine ONE person doing all three for 50 tables — chaos, and impossible to replace or train anyone.

Why this matters practically:

- 1. **Change isolation** — switching MySQL → MongoDB touches only the repository layer. The waiter and chef never learn about it.
- 2. **Testability** — you can test the chef's recipes (service logic) without a dining room (HTTP) or a storage room (real DB) — just hand him fake ingredients (mock repository — Hour 1's DI making this possible!).
- 3. **Readability** — anyone joining the project knows exactly where to look for what.

🗣️ INTERVIEW CHECKPOINT

Q: "Why shouldn't business logic live in the controller?" 🧠 "Controllers are the HTTP boundary — mixing business rules into them makes the logic untestable without HTTP, unreusable from other entry points (schedulers, message listeners), and violates single responsibility. Controllers should stay thin: receive, delegate, respond."

Q: "What does @Service actually do — is it different from @Component?" 🧠 "Functionally it's the same as @Component — it registers a bean. The layer-specific names are for readability and intent; @Repository additionally translates database exceptions into Spring's DataAccessException hierarchy." (That @Repository extra is a favorite follow-up — remember it.)

2.3 REST — the rules of the conversation

Your API and its clients need a shared convention. **REST** (REpresentational State Transfer) is that convention. Strip the jargon and REST says three simple things:

- ① **Everything is a resource, named by a noun URL.** `/api/users` (the collection), `/api/users/1` (one user). Notice: **nouns, not verbs**. Not `/getUser` or `/deleteUser` — the URL names the *thing*, not the action.
- ② **The action comes from the HTTP method.** The same URL means different operations depending on the verb:

HTTP Method	Meaning	Example	CRUD
GET	read, never change anything	GET /api/users/1	Read
POST	create new	POST /api/users	Create
PUT	replace the WHOLE resource	PUT /api/users/1	Update (full)
PATCH	update PART of it	PATCH /api/users/1	Update (partial)
DELETE	remove	DELETE /api/users/1	Delete

Analogy: the URL is the person's address; the HTTP method is what you do at that address — visit them (GET), move a new person in (POST), replace the entire family (PUT), renovate one room (PATCH), evict (DELETE). Same address, different actions.

- ③ **Stateless.** The server remembers nothing between requests. Every request must carry everything needed to serve it (like the auth token). This is what lets you scale to many servers — any server can handle any request, because no server holds "your" session. (Remember the RAG scaling hour? Load balancers spreading traffic — statelessness is what makes that possible. Full circle!)

🗣️ INTERVIEW CHECKPOINT — the REST classics

Q: "PUT vs PATCH?" 🧠 "PUT replaces the entire resource — you send the full object; missing fields are treated as removed. PATCH sends only the fields to change. PUT is idempotent by definition; PATCH may or may not be."

Q: "PUT vs POST?" / "What is idempotency?" ← extremely common pair 🧠 "An operation is idempotent when doing it once or ten times leaves the same result. PUT /users/1 ten times = still that one user in that final state → idempotent. POST /users ten times = ten new users created → not idempotent. GET and DELETE are idempotent; POST is not." Memory hook: pressing a lift button (PUT — pressing 10 times ≡ once) vs ordering a pizza (POST — 10 orders = 10 pizzas at your door).

Q: "Why is REST stateless, and why does it matter?" 🧠 "No client session lives on the server, so every request is self-contained. That makes horizontal scaling trivial — any instance behind the load balancer can serve any request — and failures don't lose session

state."

2.4 The annotations — decoding your handbook's controller

Now the code from page 6 of your handbook, understood line by line:

```
@RestController // ①
@RequestMapping("/api/users") // ②
public class UserController {

    @GetMapping("/{id}") // ③
    public User getUser(@PathVariable Long id) { // ④
        return userService.getUser(id);
    }

    @GetMapping // ⑤ GET /api/users?city=Pune
    public List<User> search(@RequestParam String city) { // ⑥
        return userService.findByCity(city);
    }

    @PostMapping // ⑦
    public User createUser(@RequestBody User user) { // ⑧
        return userService.create(user);
    }
}
```

① **@RestController** = **@Controller** + **@ResponseBody**. Translation: "I'm a bean that handles web requests (**@Controller**), AND whatever my methods return should be written directly into the response body as JSON (**@ResponseBody**) — not treated as the name of an HTML page to render." Plain **@Controller** is for old-style websites returning HTML views; **@RestController** is for APIs returning data.

② **@RequestMapping("/api/users")** at class level = the base URL. Every method's path is appended to this. Keeps you from repeating `/api/users` five times.

③ **@GetMapping("/{id}")** = handle GET requests at `/api/users/{id}`. The `{id}` part is a **placeholder** — `/api/users/1`, `/api/users/99`, all match.

④ **@PathVariable** = "pluck the value out of the URL path and hand it to me as this parameter." `/api/users/1` → `id = 1`.

⑤ **@RequestParam** = "read it from the query string" — the part after `?`. `/api/users?city=Pune` → `city = "Pune"`.

⑥ **@RequestBody** = "the data is in the request BODY as JSON — deserialize it into this object." This is Jackson working in reverse: JSON → Java object. Used with POST/PUT, where the client sends a whole object.

The three "where is the data" annotations side by side — this exact comparison is a very common question:

Annotation	Data lives in...	Example request	Typical use
@PathVariable	the URL path	<code>/users/1</code>	identifying WHICH resource
@RequestParam	the query string	<code>/users?city=Pune&page=2</code>	filtering, searching, pagination
@RequestBody	the request body (JSON)	POST with <code>{"name": "Aman"}</code>	sending whole objects to create/update

Memory hook: *PathVariable* = the house number (identifies which house). *RequestParam* = filters you tell the broker ("2BHK, under 20k"). *RequestBody* = the full moving-truck of furniture you send INTO the house.

💬 INTERVIEW CHECKPOINT

Q: "@Controller vs @RestController?" 🧠: "**@RestController** = **@Controller** + **@ResponseBody**: return values are serialized straight to the response body as JSON instead of being resolved as view names. Use **@RestController** for REST APIs."

Q: "@PathVariable vs @RequestParam?" 🧠: "**PathVariable** extracts values from the URL path itself and identifies the resource; **RequestParam** reads query-string parameters, typically optional filters or pagination. `/users/5` vs `/users?city=Pune`."

Q: "How does JSON become a Java object and back?" 🧠: "Jackson, auto-configured by Spring Boot. **@RequestBody** triggers deserialization JSON→object; return values from **@RestController** methods are serialized object→JSON."

2.5 Status codes + ResponseEntity — speaking proper HTTP

Returning data isn't enough; a good API also returns the right **status code** — the number that tells the client *how* it went. The ones you

must know cold:

Code	Meaning	When
200 OK	success	GET/PUT succeeded
201 Created	new resource born	after a successful POST
204 No Content	success, nothing to return	after DELETE
400 Bad Request	client sent garbage	validation failed
401 / 403	who are you? / you can't do that	auth / permission
404 Not Found	no such resource	GET /users/999 that doesn't exist
500 Internal Server Error	WE crashed	unhandled exception

Memory hook: 2xx = "all good", 4xx = "YOUR fault, dear client", 5xx = "MY fault, the server". Interviewers love asking "user not found — 404 or 400?" → 404: the request was well-formed; the resource just doesn't exist.

But a plain `return user;` always sends 200. To control the code, wrap the response in **ResponseEntity** — an envelope holding body + status + headers:

```
@PostMapping
public ResponseEntity<User> create(@RequestBody User user) {
    User saved = userService.create(user);
    return ResponseEntity.status(HttpStatus.CREATED).body(saved);    // 201, not 200
}

@GetMapping("/{id}")
public ResponseEntity<User> getUser(@PathVariable Long id) {
    return userService.find(id)
        .map(ResponseEntity::ok)                // found → 200 + user
        .orElse(ResponseEntity.notFound().build()); // missing → 404, no body
}
```

💬 INTERVIEW CHECKPOINT

Q: "What status should a successful POST return?" → 🧠: "201 Created — ideally with a Location header pointing at the new resource. Returning 200 for creation is a common API smell."

Q: "What is ResponseEntity and why use it?" → 🧠: "A full HTTP response wrapper — body plus status plus headers — giving precise control instead of the default 200 for everything."

(And when the interviewer asks "what about errors like user-not-found across the WHOLE app?" — that's `@RestControllerAdvice` + `@ExceptionHandler`, our Hour 7. You already know the concept from the Java guide!)

2.6 Narrate the whole hour in 30 seconds (your interview story)

Practice this until smooth — it chains everything:

"A request hits the embedded Tomcat, which passes it to the DispatcherServlet — the front controller. It routes by handler mapping to my controller method; `@PathVariable/@RequestParam/@RequestBody` extract the inputs. The controller stays thin and delegates to the service, which owns business logic and calls the repository for data. The returned object is serialized to JSON by Jackson, and I use `ResponseEntity` to send the correct status — 201 for creation, 404 when the resource doesn't exist."

That paragraph, delivered calmly, is a hire signal.

🔧 HOUR 2 DRILL — out loud

1. What is the DispatcherServlet? (front controller — reception desk)
2. Walk me through a request's journey. (the 30-second story above)
3. Why layer Controller/Service/Repository? (waiter/chef/storekeeper + change isolation, testability)
4. `@Component` vs `@Service` vs `@Repository`? (same bean registration; names = intent; `@Repository` adds exception translation)
5. PUT vs PATCH? PUT vs POST + idempotency? (lift button vs pizza order)
6. `@PathVariable` vs `@RequestParam` vs `@RequestBody`? (house number / broker filters / moving truck)
7. `@Controller` vs `@RestController`? (+`@ResponseBody`, JSON not views)
8. Status for: created? deleted? not found? validation failure? (201 / 204 / 404 / 400)

9. Why is statelessness good? (any server can serve any request → horizontal scaling)

Next → Hour 3: Inside the Container — annotations in depth, what happens when TWO beans match one @Autowired (@Qualifier/@Primary), bean lifecycle, and singleton vs prototype scope.